ESCUELA COLOMBIANA DE INGENIERÍA JULIO GARAVITO

Desarrollo Orientado por Objetos (DOPO) | Periodo 2026-2

INFORME CONSOLIDADO FINAL DE RETROSPECTIVAS

Evolución Incremental del Simulador Slot Machine y Solución Problem I (Ciclos 1, 2, 3, 4 y 5)

Equipo de Desarrollo:	Juan David Espitia y Cristian Salamanca	**Dedicación por Ciclo:**	16 Horas/Hombre (8 H/H por integrante)
Contexto del Proyecto:	Problem I - 49th ICPC World Championship	**Dedicación Consolidada:**	**80 Horas/Hombre acumuladas** (5 ciclos)

1. PRESENTACIÓN Y VISIÓN INCREMENTAL DEL PROYECTO

El presente informe documenta de manera reflexiva y analítica la totalidad del ciclo de vida del proyecto **Slot Machine**, estructurado bajo una metodología evolutiva, iterativa e incremental a lo largo de cinco ciclos académicos. El desafío central tomó como referencia el *Problem I: Slot Machine* de la final mundial de la Maratón de Programación ICPC 2025 y lo tradujo en un proyecto completo de ingeniería de software orientada por objetos.

El proyecto maduró secuencialmente: inició con el modelado del dominio y el renderizado pedagógico de figuras en BlueJ (Ciclo 1), incorporó operaciones mecánicas con dualidad transparente entre animación visual y modo invisible (Ciclo 2), formuló un algoritmo interactivo de caja negra para resolver la maratón mundial con simulación paso a paso (Ciclo 3), expandió la arquitectura mediante herencia y polimorfismo puro con cinco tipos nuevos de ruedas y símbolos (Ciclo 4), y culminó en la optimización de calidad con análisis dinámico de cobertura en **JaCoCo** (>91 %) y análisis estático en **PMD** (0 errores de prioridad 1) bajo Visual Studio Code (Ciclo 5).

Ciclo 1: Dominio Base
Modelado inicial en Astah, desacoplamiento GUI con Shapes, operaciones básicas en BlueJ.

Ciclo 2: Extensión & TDD
Swap, lock/unlock, animación paso a paso vs. ejecución atómica invisible, pruebas JUnit 4.

Ciclo 3: Algoritmia ICPC
SlotMachineContest, deducción a ciegas en ≤ 10.000 jugadas y simulación visual ganadora.

Ciclo 4: Polimorfismo
Jerarquía de Ruedas (Lefty, Rebel, Inverted) y Símbolos (Ephemeral, Shy) sin romper contratos.

Ciclo 5: Calidad Total
Migración a VS Code, JaCoCo (73.5% → 91.2%), PMD (13 rojas → 0) y arnés de aceptación.

Tabla Resumen de Evolución Incremental (Ciclos 1 al 5)

Ciclo	Enfoque Principal	H/H	Mayor Logro Técnico y Arquitectónico	Desafío Crítico Superado
Ciclo 1	Modelado de Dominio y GUI Shapes	16 h	Arquitectura desacoplada: `SlotMachine` opera como orquestador sin contaminarse de coordenadas gráficas.	Encapsular dibujo relativo en cada rueda y detener trazas de Astah en la frontera de `shapes`.
Ciclo 2	Extensión Mecánica y Animación	16 h	Dualidad transparente en `spin(wheel, steps)`: animación suave visible y ejecución atómica en memoria.	Eliminar redundancia de giros con método atómico `rotateOneStep()` y modelar fragmentos UML (alt/loop/opt).
Ciclo 3	Maratón ICPC y Caja Negra	16 h	Resolución a ciegas del Problem I en ≤ 10.000 jugadas acoplada a simulación visual con `simulate(n)`.	Garantizar caja negra estricta en `SlotMachineContest` sin vulnerar campos privados del simulador.
Ciclo 4	Herencia y Polimorfismo	16 h	Inclusión de 5 tipos nuevos (Lefty, Rebel, Inverted, Ephemeral, Shy) preservando contratos previos intactos.	Trasladar lógica especializada a subclases para mitigar desorden acumulado de ciclos anteriores.
Ciclo 5	Calidad con JaCoCo y PMD	16 h	Cobertura de 73.51 % a 91.22 % (38 pruebas en verde) y 100 % de reglas críticas de PMD corregidas (13 a 0).	Conciliar reglas de PMD con restricciones BlueJ y Astah, renombrando formalmente `solveBase`.

2. RETROSPECTIVA CICLO 1: CONSTRUCCIÓN INICIAL DEL SIMULADOR Y BASE EN SHAPES

Alcance: Modelado del dominio en BlueJ, gestión de ruedas y símbolos, rotación, detección de jackpot, alternancia visible/invisible y usabilidad con el paquete pedagógico shapes.

1. ¿Cuáles fueron los mini-ciclos definidos? Justifíquenlos.

- **MC 1.1 Modelado del Dominio y Diseño en Astah:** Diagrama de clases inicial definiendo `SlotMachine`, `Wheel` y `Symbol`. *Justificación:* Asignar responsabilidades con alta cohesión antes de codificar, impidiendo que la máquina absorbiera tareas propias de las ruedas.
- **MC 1.2 Lógica del Requerimiento: Ruedas y Símbolos (RF1 - RF3):** Creación de la tragamonedas, adición/eliminación de ruedas (`addWheel`, `delWheel`) y gestión de símbolos diferenciados por color (`addSymbol`, `delSymbol`). *Justificación:* Validar la coherencia del estado interno en memoria (límites e invariantes) con independencia de la GUI.
- **MC 1.3 Dinámica de Juego y Detección de Jackpot (RF4 - RF6):** Mecanismos de giro (`spin`), consulta de configuraciones actuales (`symbols`, `configuration`) y algoritmo de victoria (`isJackpot`). *Justificación:* Verificar matemáticamente el desplazamiento modular de índices de forma aislada.
- **MC 1.4 Integración Gráfica y Usabilidad con Shapes (RF7 - RF8):** Alternancia de visibilidad (`makeVisible`, `makeInvisible`), salida (`exit`) y retroalimentación modal con `JOptionPane` condicionada estrictamente a `isVisible`. *Justificación:* Aislar el renderizado al final protegió la lógica del dominio, asegurando que la máquina fuera plenamente funcional en modo invisible.

2. Estado del proyecto en términos de mini-ciclos

Mini-ciclo	Nombre	Estado	Evidencias y Verificación del Estado
MC 1.1	Modelado Astah	**85 % Completado**	Diagrama de clases completo; diagramas de secuencia deteniendo la traza formalmente en shapes.
MC 1.2	Lógica Ruedas/Símbolos	**100 % Completado**	Métodos `addWheel`, `delWheel`, `addSymbol` y `delSymbol` operativos gestionando límites e índices.
MC 1.3	Dinámica y Jackpot	**100 % Completado**	Rotación circular de `spin()` validada; métodos de consulta e `isJackpot()` verificados.
MC 1.4	Integración Shapes & GUI	**100 % Completado**	Alternancia visual funcional; ventanas modales `JOptionPane` desplegadas solo cuando la máquina está visible.

Dedicación:	Juan David Espitia (8h - Arquitectura y Dominio) | Cristian Salamanca (8h - GUI y Usabilidad)

4. Mayor logro, problemas técnicos y compromisos

> **Mayor Logro:** Arquitectura desacoplada. El núcleo de `SlotMachine` opera con absoluta independencia del motor de dibujo de `shapes`. Esta separación fue el cimiento que permitió agregar pruebas automatizadas en el Ciclo 2 y el testing tool en el Ciclo 3 sin necesidad de reescribir código.

Mayor Problema Técnico: Desalineación de coordenadas relativas de figuras y llamadas indebidas a `Canvas` en modo invisible. *Solución:* Encapsular el renderizado dentro de cada entidad del dominio, calculando posiciones relativas y subordinando cualquier dibujo a la bandera booleana `isVisible`. En Astah, se modelaron las clases de shapes como *boundary objects*, deteniendo la traza en sus métodos públicos.

Compromiso asumido para el Ciclo 2: Abandonar la dependencia de pruebas manuales en el banco de objetos de BlueJ e incorporar desarrollo guiado por pruebas unitarias automatizadas con JUnit 4.

3. RETROSPECTIVA CICLO 2: REFACTORING, EXTENSIÓN Y PRUEBAS UNITARIAS

Alcance: Extensión funcional (`swap`, `lock/unlock`, `spin` parametrizado, `set`), animación paso a paso y suite de pruebas unitarias automatizadas en modo invisible.

1. Mini-ciclos definidos y Estado de Cumplimiento

- **MC 2.1 Refactorización y Diseño de Extensión en Astah (80 %):** Incorporación de `swap`, `lock`, `unlock` y variantes de giro con diagramas de secuencia dinámicos.
- **MC 2.2 Implementación de Extensiones Funcionales (100 %):** Lógica transaccional asegurando que una rueda bloqueada rechace cualquier rotación y que `swap` preserve referencias.
- **MC 2.3 Visualización Dinámica y Animación Paso a Paso (100 %):** Animación frame a frame en `spin(wheel, steps)` con temporizador en Canvas al estar visible, y ejecución atómica en modo invisible.
- **MC 2.4 Pruebas Unitarias Automatizadas (80 %):** Construcción de la suite `SlotMachineC2Test` en modo invisible ("qué debería hacer" vs "qué no debería hacer") y aportes a `SlotMachineCC2Test`.

> **Mayor Logro:** Dualidad operativa transparente en `spin(wheel, steps)`. Lograr que el simulador ofrezca una experiencia visual fluida paso a paso en pantalla, pero ejecute de manera instantánea y atómica en modo invisible, permitiendo que las pruebas automatizadas corran en milisegundos.

Mayor Problema Técnico: Duplicación de lógica entre las cuatro sobrecargas de `spin`. *Solución:* Refactorización modular extrayendo el método atómico `rotateOneStep()` en `Wheel`, el cual centraliza la validación de bloqueo y el avance modular de punteros. En Astah se aplicaron fragmentos combinados `alt` (bloqueo), `loop` (iteraciones) y `opt` (animación subordinada a visibilidad).

Compromiso asumido para el Ciclo 3: Diseñar y formalizar la solución matemática del Problem I de la ICPC en papel antes de programar, asegurando desacoplamiento estricto frente a la testing tool.

4. RETROSPECTIVA CICLO 3: SOLUCIÓN MARATÓN MUNDIAL ICPC Y SIMULACIÓN VISUAL

Alcance: Desacoplamiento de `SlotMachine` como testing tool de caja negra y simulador, resolución interactiva del Problem I (ICPC World Finals 2025) y simulación visual.

1. Mini-ciclos definidos y Estado de Cumplimiento

- **MC 3.1 Arquitectura y Diseño en Astah (90 %):** Modelado de la controladora `SlotMachineContest` como cliente externo ciego respecto a `SlotMachine`.
- **MC 3.2 Extensión del Simulador para Maratón (100 %):** Constructor `SlotMachine(n)` aleatorio no ganador y método oráculo `distinctSymbols()` (*k*).
- **MC 3.3 Algoritmo de Solución ICPC (100 %):** Implementación de `solve(n): int[][]` deduciendo a ciegas la alineación de ruedas en ≤ 10.000 acciones mediante la retroalimentación de *k*.
- **MC 3.4 Simulación Visual de la Secuencia Ganadora (100 %):** Método `simulate(n)` reproduciendo paso a paso sobre la máquina visible la matriz calculada por `solve`.
- **MC 3.5 Verificación Sistemática y Pruebas Compartidas (90 %):** Suite `SlotMachineContestTest` y colaboración en `SlotMachineContestCC2Test`.

> **Mayor Logro:** Solución formal de un problema de la final mundial de la ICPC bajo principios puros de orientación a objetos. Se demostró que la disciplina de diseño (caja negra estricta por composición) potencia la algoritmia compleja y permite articular inmediatamente la solución matemática con una simulación gráfica sincronizada.

Mayor Problema Técnico: Coexistencia de roles opuestos en `SlotMachine` (evaluador silencioso ciego vs simulador gráfico). *Solución:* Aislamiento estricto por composición: `solve(n)` consume únicamente la interfaz autorizada en modo invisible y `simulate(n)` reutiliza la matriz resultante para activar el renderizado con `makeVisible()`. En Astah se elaboraron dos diagramas de secuencia independientes para no sobrecargar el modelo.

Compromiso asumido para el Ciclo 4: Extender el simulador incorporando tipos polimórficos de ruedas y símbolos sin alterar la compatibilidad con el solucionador de la maratón.

5. RETROSPECTIVA CICLO 4: REFACTORING Y EXTENSIÓN CON TIPOS DE RUEDAS Y SÍMBOLOS

Alcance: Jerarquías de especialización de ruedas (`normal`, `lefty`, `rebel`, `inverted`) y símbolos (`normal`, `ephemeral`, `shy`), creación por tipo desde `SlotMachine`, validaciones y pruebas de aceptación.

1. Mini-ciclos definidos y Estado de Cumplimiento

- **MC 4.1 Cierre y Extensión del Diseño en Astah (100 %):** Modelado formal de las jerarquías de herencia y sobrecargas `addWheel(type, pos)` y `addSymbol(type, pos, color)`.
- **MC 4.2 Jerarquía de Ruedas (90 %):** `Lefty` (copia a la vecina izquierda), `Rebel` (inmune a bloqueo, intercambio y eliminación) e `Inverted` (tipo propuesto por el equipo: invierte el giro por pasos).
- **MC 4.3 Jerarquía de Símbolos (100 %):** `Ephemeral` (reduce tamaño progresivamente) y `Shy` (alterna visibilidad), con `copy()` polimórfico.
- **MC 4.4 Métodos Nuevos en SlotMachine (90 %):** Creación centralizada por tipo preservando las firmas de ciclos anteriores mediante delegación.
- **MC 4.5 Manejo de Errores y Validaciones (90 %):** Mensajes con `SlotMachineDialog` condicionados a la visibilidad activa.
- **MC 4.6 Pruebas Unitarias y Aceptación (90 %):** Suite `SlotMachineC4Test` (13 casos) y casos compartidos en `SlotMachineCC4Test`.

> **Mayor Logro:** Extensibilidad limpia mediante herencia y polimorfismo puro. Se agregaron cinco tipos nuevos sin tocar la lógica de `SlotMachineContest` ni alterar firmas públicas previas. Se introdujo la rueda `Inverted` como propuesta innovadora del equipo, resuelta con una sola redefinición modular de `spin(steps)`.

Mayor Problema Técnico: Desorden acumulado de ciclos anteriores (código repetido de reposicionamiento, refresco visual y reasignación de enlaces de Lefty). *Solución:* Trasladar el comportamiento propio a las subclases especializadas, dejando catalogada la extracción de métodos privados como deuda técnica para el Ciclo 5.

Compromiso asumido para el Ciclo 5: Migrar a VS Code y evaluar la calidad del código mediante herramientas estándar de la industria (JaCoCo para cobertura dinámica y PMD para análisis estático).

6. RETROSPECTIVA CICLO 5: CALIDAD DEL SIMULADOR CON ANÁLISIS DINÁMICO Y ESTÁTICO

Alcance: Migración a Visual Studio Code, cierre y actualización del diseño en Astah, análisis dinámico con JaCoCo, análisis estático con PMD, refactorización derivada de los hallazgos, robustecimiento de pruebas y prueba de aceptación.

1. ¿Cuáles fueron los mini-ciclos definidos? Justifíquenlos.

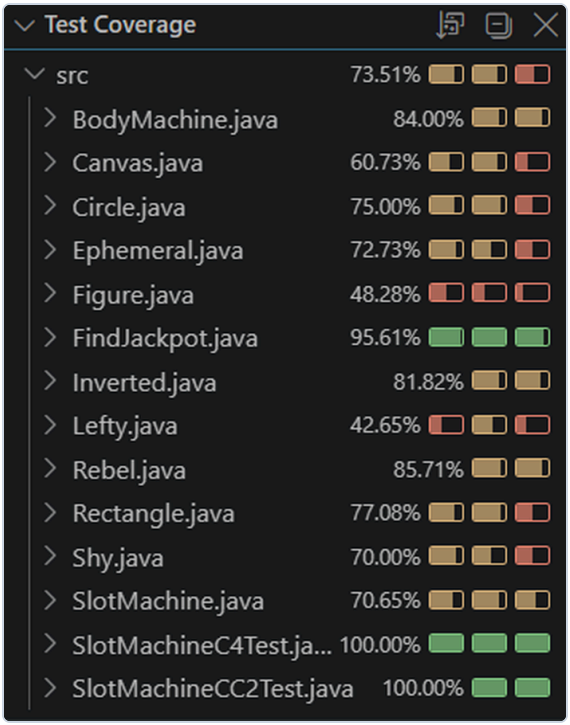
El Ciclo 5 no agregó funcionalidad de negocio nueva; su propósito fue elevar el estándar de calidad del software. Se estructuraron seis mini-ciclos para desacoplar el diagnóstico, la refactorización y la verificación:

- **MC 5.1 Migración a VS Code (100 %):** Traslado a VS Code con *Extension Pack for Java*; configuración de librerías en `lib/` y sincronización de la carpeta `slotMachineBlueJ` para mantener compatibilidad dual.
- **MC 5.2 Diseño en Astah (90 %):** Actualización con la interfaz `FindJackpot`, `SlotMachineContest` y reflejo del renombrado formal `solveBase`.
- **MC 5.3 Análisis Dinámico con JaCoCo (100 %):** Medición inicial (73.51 %), detección de brechas críticas, adición de 7 pruebas unitarias y elevación de la cobertura al **91.22 %** (meta >75 %).
- **MC 5.4 Análisis Estático con PMD (90 %):** Ejecución con ruleset `quickstart.xml` y corrección de la totalidad de las **13 alertas de prioridad 1** (errores críticos).
- **MC 5.5 Refactorización y Limpieza (90 %):** Variables a *camelCase*, métodos con modificadores correctos, clases `final` en constructores privados y eliminación de código muerto.
- **MC 5.6 Pruebas Unitarias y Aceptación (90 %):** 38 pruebas unitarias consolidadas en verde y suite visual interactiva `SlotMachineAcceptanceTest` (91.67 % de cubrimiento).

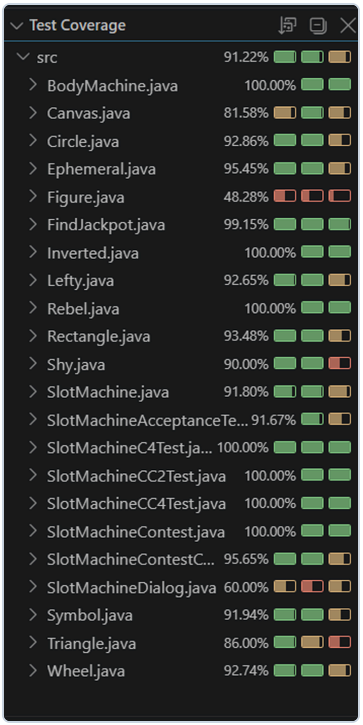
4. Mayor Logro: Elevación Medible de la Calidad (JaCoCo y PMD)

> **Mayor Logro del Ciclo 5:** Superar contundentemente las dos metas formales de calidad sin introducir regresiones sobre los ciclos anteriores: el cubrimiento dinámico de pruebas subió de **73.51 % a 91.22 %** y las alertas de prioridad 1 de PMD pasaron de **13 a 0**, manteniendo **las 38 pruebas unitarias en verde**.

Evidencia Visual del Salto de Cobertura Dinámica (JaCoCo):



Línea Base Inicial de JaCoCo: 73.51 %



Resultado Final de JaCoCo: 91.22 % (Meta Superada)

Cuadro Comparativo de Cobertura JaCoCo por Clase:

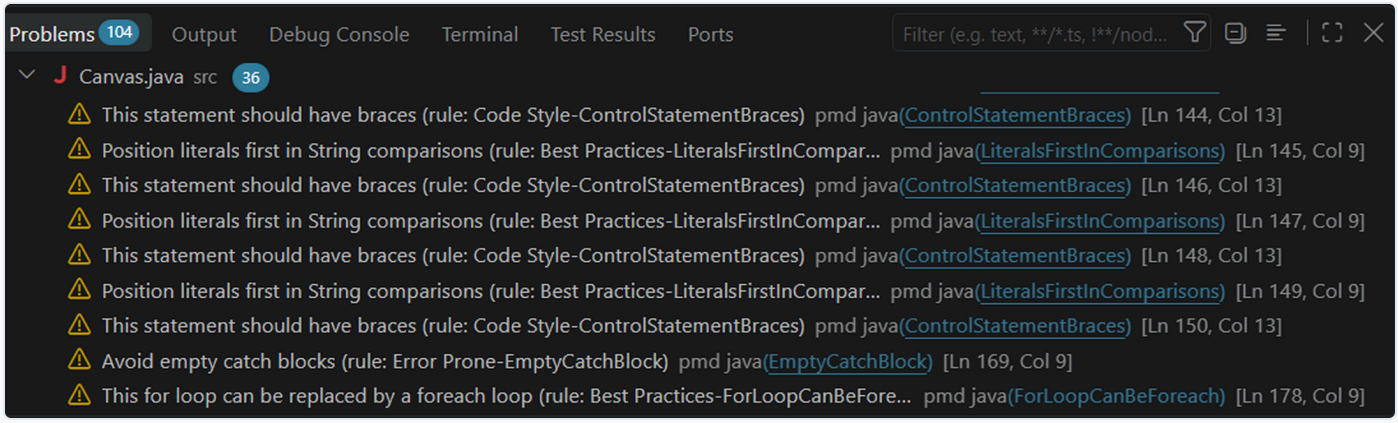
Clase	Cobertura Inicial	Cobertura Final	Incremento	Diagnóstico
`BodyMachine`	84.00 %	**100.00 %**	+16.00 %	Totalmente cubierta
`Canvas`	60.73 %	**81.58 %**	+20.85 %	Supera meta (>75 %)
`Circle`	75.00 %	**92.86 %**	+17.86 %	Alta cobertura
`Ephemeral`	72.73 %	**95.45 %**	+22.72 %	Alta cobertura
`Figure`	48.28 %	**48.28 %**	0.00 %	Clase abstracta gráfica
`FindJackpot`	95.61 %	**99.15 %**	+3.54 %	Prácticamente total
`Inverted`	81.82 %	**100.00 %**	+18.18 %	Totalmente cubierta
`Lefty`	42.65 %	**92.65 %**	**+50.00 %**	Mayor incremento del proyecto
`Rebel`	85.71 %	**100.00 %**	+14.29 %	Totalmente cubierta
`Rectangle`	77.08 %	**93.48 %**	+16.40 %	Alta cobertura
`Shy`	70.00 %	**90.00 %**	+20.00 %	Alta cobertura
`SlotMachine`	70.65 %	**91.80 %**	+21.15 %	Supera meta (>75 %)
`SlotMachineContest`	63.64 %	**100.00 %**	+36.36 %	Totalmente cubierta
`SlotMachineDialog`	25.00 %	**60.00 %**	+35.00 %	Modo visible condicionado
`Symbol`	91.94 %	**91.94 %**	0.00 %	Estable y sólida
`Triangle`	68.00 %	**86.00 %**	+18.00 %	Supera meta (>75 %)
`Wheel`	65.86 %	**92.74 %**	+26.88 %	Alta cobertura
TOTAL CONSOLIDADO	**73.51 %**	**91.22 %**	**+17.71 %**	**Meta ampliamente cumplida**

5. Mayor Problema Técnico del Ciclo 5 y Solución Aplicada

El reto principal consistió en conciliar las recomendaciones del analizador estático PMD con las restricciones del entorno educativo (estructura sin paquetes para BlueJ y fidelidad con los diagramas de clases en Astah). De las 117 alertas iniciales, se eliminó el 100 % de las 13 alertas críticas de Prioridad 1:

- **10 alertas en SlotMachine:** Nombres de variables locales en *snake_case* (`principal_body`, `max_wheel`, `body_x`, etc.) convertidas estrictamente a *camelCase*.
- **2 alertas en Canvas y CanvasPane:** Clases con constructores exclusivamente privados que no eran `final`; se aplicó el modificador `final` garantizando inmutabilidad estructural.
- **1 alerta en FindJackpot:** Método `solve_base` renombrado formalmente a `solveBase` en la interfaz, en `SlotMachineContest`, en las pruebas y en los diagramas de Astah.

Evidencia Visual del Análisis Estático (PMD en Visual Studio Code):



Panel Problems en VS Code: 0 alertas de prioridad 1; se conservan deliberadamente las 104 sugerencias menores de prioridad 3 en Canvas.java

7. MATRIZ DE TRAZABILIDAD EVOLUTIVA GLOBAL (CICLOS 1 AL 5)

Dimensión	Ciclos 1 y 2 (Fundación)	Ciclos 3 y 4 (Algoritmo & Polimorfismo)	Ciclo 5 (Calidad Profesional)
Entorno	BlueJ nativo, banco de objetos y compilación plana.	BlueJ y modelado exhaustivo en Astah.	Dualidad completa: VS Code (JaCoCo, PMD, Git) + carpeta nativa `slotMachineBlueJ`.
Arquitectura	`SlotMachine` orquestador desacoplado de `shapes` con guarda `isVisible`.	Caja negra en `SlotMachineContest`. Jerarquías polimórficas de ruedas y símbolos.	Consolidación de contratos (`FindJackpot`), inmutabilidad en GUI y código limpio.
Pruebas	Pruebas manuales → Primera suite `SlotMachineC2Test` (JUnit 4).	Pruebas de oráculo determinístico y suites de tipos (`C4Test`, `CC4Test`).	**38 pruebas unitarias en verde** con **91.22 % de cobertura JaCoCo** y aceptación visual.
Modelado Astah	Clases de dominio y secuencias acotadas en la frontera de shapes.	Secuencias dinámicas con fragmentos combinados (alt/loop/opt) y jerarquías.	Sincronización de interfaces, normalización de firmas (`solveBase`) y clases de prueba.
Calidad	Convenciones básicas y métodos menores a una pantalla.	Métodos atómicos (`rotateOneStep`) y polimórficos (`copy`).	**0 alertas rojas PMD**, estándar *camelCase* y gestión transparente de deuda técnica.

8. COMPROMISOS PARA LA SUSTENTACIÓN FINAL DEL PROYECTO

Como cierre integral del proyecto y en preparación para la sustentación definitiva ante el jurado docente, el equipo de desarrollo asume los siguientes compromisos técnicos:

1. **Gestión de Tipos Desconocidos:** Parametrizar `addWheel` y `addSymbol` para que emitan advertencias explícitas ante tipos no registrados en lugar de instanciar elementos normales silenciosamente.
2. **Refactorización de Métodos Privados en SlotMachine:** Extraer a métodos privados auxiliares la lógica duplicada de reposicionamiento y refresco gráfico, reemplazando comparaciones de cadenas por un mecanismo de creación por tipo y reflejándolo en Astah.
3. **Atención a Sugerencias de Prioridad 3 de PMD:** Incorporar llaves en bloques de control, literales primero en comparaciones y bucles `foreach` en `Canvas.java`.
4. **Cobertura Remanente:** Ampliar pruebas en `Figure.java` (actualmente en 48.28 %) ejercitando el comportamiento base por defecto en modo invisible.
5. **Demostración Dual en Sustentación:** Demostrar la funcionalidad tanto en Visual Studio Code mediante la prueba de aceptación interactiva como en BlueJ a través del banco de objetos.
6. **Estado de Casos Extremos:** Dejar formalmente documentado que la cota algorítmica para $n = 50$ permanece como trabajo futuro de optimización heurística, estando el sistema 100 % probado para $n \in [1, 5]$.
